Study book

Topic 7: DI, middleware, configuration, ILogger

ProjectHub learning track: React/TypeScript to C#/.NET backend. Week 2, topic 7 of 20.

Brief overview

What this lesson is about. How an ASP.NET Core app is wired: **dependency injection** (the container, registrations, **Singleton vs Scoped vs Transient**, captive dependencies), the **middleware pipeline** (our own request-timing middleware), **configuration** (appsettings, environments, environment variables, the Options pattern, fail-fast validation) and **structured logging** with `ILogger`.
What we built. `ProjectStoreOptions` (simulated latency as a setting), `RequestTimingMiddleware` (header + one log line per request), `ILogger` in `ProjectService`, a factory registration in `Program.cs`, and the **lifetimes lab** `GET /api/labs/lifetimes`.
Checkpoint: explain Scoped vs Transient vs Singleton — proven with real ids from the lab, run by you and in the cloud.
Your questions shaped this book: composition, how controllers are discovered, generic type arguments, the container itself, anonymous objects, and a debugger puzzle (6 breakpoint stops) where you were right and I was wrong.

Contents

Part	What is inside
1	DI: what, why, the container (your question), composition vs inheritance (your question)
2	Registering and resolving: two registration styles, the container-created rule, generics as configuration
3	The three lifetimes (the checkpoint), the lab, captive dependencies
4	Middleware: the pipeline, our timing middleware, OnStarting, order
5	Controller discovery: AddControllers + MapControllers (your question)
6	Configuration: sources and precedence, Options pattern, fail fast
7	ILogger: categories, levels, message templates
8	Our code: Program.cs, ProjectService, the lab, anonymous objects (your question)
9	Real runs: lifetimes, logs, headers, env var override, bad config, captive dependency
10	Debugging: the 6-stops puzzle, creations vs reads, breaking the rules
11	Your answers, corrections, interview Q&A
12	Summary, cheat sheet, checklist, what comes next

Part 1. Dependency injection

1.1 What and why

```
// WITHOUT DI: the class creates what it needs (hard-wired)
public class ProjectsController
{
    private readonly ProjectService _projects = new ProjectService(SampleData.CreateProjects(), ...);
}

// WITH DI: the class only DECLARES what it needs; someone else provides it
public class ProjectsController(IProjectService projects) : ControllerBase
```

- Without DI the controller decides the concrete class, its constructor arguments and its lifetime; you cannot swap it, fake it in a test, or share one store.
- With DI: **registration** once in `Program.cs` (the **composition root**), **resolution** by the container when an object is needed: **constructor injection**, **inversion of control**.
- **Benefits (name two in the interview):** swap implementations (in-memory -> EF Core in topic 10 with one line), fakes in tests (topic 13), lifetimes managed in one place.

1.2 What IS the container? (your question)

You asked: 'container is what? Program? passes them into its constructor — what, where? All I see is we declare in Program and then just have them in constructors of deeply nested classes.'

1. **Program.cs = the recipe book.** `builder.Services.AddScoped<LifetimeReporter>()` writes a recipe; nothing is created yet.
2. `builder.Build()` **turns the recipes into the container**: a `ServiceProvider` object in memory inside the running app. The `serviceProvider` in the factory registration **is** the container; `GetRequiredService<...>()` asks it by hand.
3. **A request arrives.** ASP.NET asks the container for a `LifetimesController`. The container reads the constructor, creates or reuses each dependency (recursively) and **calls the constructor itself** with them as arguments.

```
// what the container does internally, roughly
var singleton = existingSingleton ?? new SingletonOperation();     // recipe: reuse
var scoped    = thisRequestsScoped ?? new ScopedOperation();       // recipe: one per request
var reporter  = new LifetimeReporter(singleton, scoped, new TransientOperation());
var controller = new LifetimesController(singleton, scoped, new TransientOperation(), reporter);
```

Remember

Your observation was exactly right: **declare in Program.cs, then just have them in constructors of deeply nested classes.** The container is the invisible thing in between that writes every `new` for you. React analogy: a `<Provider>` at the top + `useContext()` deep inside — but the container also **creates** the objects and decides **how long they live**.

1.3 Composition vs inheritance (your question)

You asked whether composition is 'anti-inheritance, so like independent'. Not quite: it is **another way to reuse code**.

	Inheritance	Composition
Relationship	**is-a**: a ProjectsController *is a* ControllerBase	**has-a**: a ProjectsController *has an* IProjectService

	Inheritance	Composition
How you get behaviour	become a subtype, inherit methods	hold another object (field / constructor parameter) and call it
Syntax	`class X : BaseClass`	`class X(IDependency dep)` then `dep.DoSomething()`

- **React chose composition** ('Composition vs Inheritance' in the React docs): components built out of components, children, props, hooks — never `extends Button`.
- Our code: `ProjectsController(IProjectService projects) : ControllerBase` (has-a + is-a); `ProjectService` has a List, a Lock, an ILogger; `LoggingProjectService` wraps an inner `IProjectService` (decorator = composition).
- **Why prefer it:** swappable parts (DI, EF, fakes), only one base class in C#, no fragile base class, small focused classes instead of deep hierarchies.
- **'Independent' is partly right:** the parts are **loosely coupled** (the controller doesn't know which implementation it holds) but it still **depends** on the part's interface — DI plugs it in.
- Inheritance stays normal for framework base classes (`ControllerBase`, `Exception`) and real is-a families.

Part 2. Registering and resolving

Your summary: 'to make DI work we register it in Program.cs, then we can use it in a constructor — not just in a controller, in any class.' Right, with one condition:

The condition

Only classes the container creates get injection. Controllers (created by ASP.NET), middleware, and other registered services. If **you** write `new LifetimeReporter(...)`, there is no DI: you pass all arguments yourself — exactly what the playground demos do with `new ProjectService(...)`. Rule: in app code, don't `new` services; ask for them in the constructor.

Style	Example	When
By type: the container calls the constructor and resolves each parameter	`AddScoped<LifetimeReporter>()`	all constructor parameters are registered services
Factory: you write the `new`	`AddSingleton<IProjectService>( sp => new ProjectService(...))`	the constructor needs non-services (`List<Project>`, `TimeSpan`)
Interface -> class (everyday form)	`AddScoped<IProjectService, ProjectService>()`	whoever asks for the interface gets that class (topic 10)

Forgotten registration: `Unable to resolve service for type 'X' while attempting to activate 'Y'` (checked at startup in Development).

2.1 A class as a generic argument (your observation)

`app.UseMiddleware<RequestTimingMiddleware>()` works because C# generics are **reified** (topic 3): the method uses `typeof(T)` at runtime, finds the constructor, asks DI for the parameters, creates the object and calls `InvokeAsync` per request. TS generics are erased, so TS libraries pass **the class as a value** (`app.use(new X(...))`, NestJS/Inversify class tokens). C# also has `app.UseMiddleware(typeof(RequestTimingMiddleware))`; the generic form is preferred.

Code	What the <T> tells it
`AddSingleton<SingletonOperation>()`	which class to register and create
`AddOptions<ProjectStoreOptions>()`	which class to fill from appsettings
`GetRequiredService<ILogger<ProjectService>>()`	which service to get from the container
`ILogger<ProjectService>`	the log category
`AddSingleton<TService, TImplementation>()`	constraint `where TImplementation : class, TService`: `AddSingleton<IProjectService, SampleData>()` does not compile

Part 3. The three lifetimes (the checkpoint)

Lifetime	Registration	One object per...	Typical use
Singleton	`AddSingleton<T>()`	**application process** (until it stops)	caches, config, our in-memory store; **must be thread-safe**
Scoped	`AddScoped<T>()`	**HTTP request**	EF Core `DbContext` (topic 10), per-request state
Transient	`AddTransient<T>()`	**injection** (every time it's asked for)	small stateless helpers

3.1 Proof: the lifetimes lab

Cloud run (you ran it yourself too, two requests in one session):

```
request 1  controller: singleton ef200b0b  scoped 222eb6c4  transient 08a150dd
           reporter:   singleton ef200b0b  scoped 222eb6c4  transient ace29ac3
request 2  controller: singleton ef200b0b  scoped 34f8a033  transient 69a01101
           reporter:   singleton ef200b0b  scoped 34f8a033  transient 9d9ee21f
```

Lifetime	Same within a request?	Same across requests?	Evidence
Singleton	yes	**yes**	`ef200b0b` everywhere
Scoped	**yes**	no	`222eb6c4` twice, then `34f8a033`
Transient	**no**	no	4 different ids

Your conclusion

Your words: 'singleton same during the run/debug session; scoped per request; transient always unique, just in a scope of a class or something'.
Precisions: singleton = lifetime of the **process** (restart -> new id; also why the in-memory projects reset on restart). Transient = **per injection**, not per class: a class asking twice gets two objects.
Interview lines: **Singleton: one object for the whole app. Scoped: one per HTTP request. Transient: a new one every time it is injected.**

3.2 How to choose

- **Scoped** for app services that use a `DbContext` (topic 10): a service can't outlive what it holds.
- **Singleton** only for truly shared, **thread-safe** things. `ProjectService` is a singleton because it *is* the in-memory store (topic 4's lock makes it safe); in topic 10 it becomes **Scoped** because it will use a `DbContext` per request.
- **Transient** for light stateless helpers.

3.3 Captive dependencies: the rule you must not break

A service may only depend on things that live at least as long as itself. We registered `LifetimeReporter` as a singleton on purpose; ASP.NET refused to start:

```
Some services are not able to be constructed ...
Cannot consume scoped service 'ProjectHub.Api.Labs.Lifetimes.ScopedOperation'
from singleton 'ProjectHub.Api.Labs.Lifetimes.LifetimeReporter'.
```

You asked (debug step 3): 'what uses the reporter so it's not ok?' — it's not **who uses** it but **what it holds**: its constructor takes a `ScopedOperation`. A singleton is created once and would keep request 1's scoped object **forever**. Real life: one request's `DbContext` or 'current user' leaking into everyone's requests.

A ... may depend on a ...	Singleton	Scoped	Transient
Singleton	yes	**no (captive)**	allowed, but lives as long as the singleton
Scoped	yes	yes	yes
Transient	yes	yes	yes

Common mistake

Your first answer: 'singleton relies on scoped **or transient**'. Transient inside a singleton is allowed (it just lives as long as the singleton). The captive problem is singleton -> **scoped**. And say what ASP.NET does: in Development it refuses to start.

Part 4. Middleware

```
request  -> RequestTiming -> HttpsRedirection -> Authorization -> [routing -> controller action]
response <- RequestTiming <- HttpsRedirection <- Authorization <-
```

Each middleware can work **before** `next`, call `next` (the rest of the pipeline), work **after**, or **short-circuit** (answer without calling `next`, e.g. auth -> 401). Express: `app.use(async (req, res, next) => { ...; await next(); ... })`.

src/ProjectHub.Api/Middleware/RequestTimingMiddleware.cs

```
using System.Diagnostics;

namespace ProjectHub.Api.Middleware;

// Measures every request: adds an "X-Response-Time-Ms" header and writes one log line per request.
// Express equivalent: app.use(async (req, res, next) => { const start = Date.now(); await next(); ... })
public class RequestTimingMiddleware(RequestDelegate next, ILogger<RequestTimingMiddleware> logger)
{
    public const string HeaderName = "X-Response-Time-Ms";

    // Called once per request. "next" is the rest of the pipeline (routing, controllers, ...).
    public async Task InvokeAsync(HttpContext context)
    {
        var stopwatch = Stopwatch.StartNew();

        // Headers can only be changed BEFORE the response starts being sent,
        // so the header is added in a callback that runs right before that moment.
        context.Response.OnStarting(() =>
        {
            context.Response.Headers[HeaderName] = stopwatch.ElapsedMilliseconds.ToString();
            return Task.CompletedTask;
        });

        try
        {
            await next(context); // everything after this middleware runs here
        }
        finally
        {
            // finally: log even if something further down threw an exception.
            logger.LogInformation(
                "HTTP {Method} {Path} responded {StatusCode} in {ElapsedMs} ms",
                context.Request.Method,
                context.Request.Path,
                context.Response.StatusCode,
                stopwatch.ElapsedMilliseconds);
        }
    }
}
```

Piece	Why
`RequestDelegate next` / `HttpContext context`	Express's `next` / `req` + `res`
`Response.OnStarting(...)`	headers go out **before** the body; after `await next` the response may already have started ('response has already started'); OnStarting runs at the last moment headers can change
`try { await next } finally { log }`	log even if something below throws (status may still say 200 then; topic 12 fixes it)
`app.UseMiddleware<...>()` **first**	it measures everything after it; **order matters**: a middleware only sees what comes after it

Piece	Why
created once	middleware behaves like a singleton: scoped services go into `InvokeAsync` parameters, not the constructor

The startup warning `Failed to determine the https port for redirect` is harmless: the http-only profile gives HTTPS redirection nowhere to redirect.

Part 5. How controllers are found (your question)

You noticed `LifetimesController` works although it is not in the `Controllers` folder. Two steps, and the folder doesn't matter:

1. `AddControllers()` **discovers** controllers by **reflection** over the compiled assembly: public, non-abstract, and (inherits `ControllerBase` OR name ends with `Controller` OR has `[Controller]`/`[ApiController]`). `Controllers/` is only a human convention.
2. `MapControllers()` **maps** each action's `[Route]` + `[HttpGet]`... into an **endpoint** (verb + URL -> action) that routing matches.

- No `MapControllers()`: every request 404. No `AddControllers()`: `MapControllers()` fails at startup asking for it.
- NestJS decorators (`@Controller`, `@Get`) are the same idea; Next.js is the opposite (the folder **is** the route).
- All discovered endpoints: `http://localhost:5064/openapi/v1.json`.

Part 6. Configuration

Order (later wins)	Source	Note
1	`appsettings.json`	defaults (ours: SimulatedLatencyMs 0)
2	`appsettings.{Environment}.json`	`ASPNETCORE_ENVIRONMENT`; launchSettings sets Development (ours: 50)
3	user secrets	Development only, outside the repo
4	**environment variables**	`ProjectStore__SimulatedLatencyMs=500` (double underscore = nesting)
5	command-line arguments	

Secrets never go in appsettings in git: user secrets locally, Key Vault in Azure (topic 19).

Common mistake

Your first answer covered appsettings -> appsettings.Development.json (right) but missed that **environment variables beat both** — how servers and Azure change settings without editing files. Re-check: right.

src/ProjectHub.Api/Configuration/ProjectStoreOptions.cs

```
using System.ComponentModel.DataAnnotations;

namespace ProjectHub.Api.Configuration;

// Strongly typed settings for the in-memory project store, bound from the "ProjectStore" section of
// appsettings*.json (or environment variables like ProjectStore__SimulatedLatencyMs=500).
public class ProjectStoreOptions
{
    public const string SectionName = "ProjectStore";

    // How long each simulated "database call" waits. 0 = instant.
    [Range(0, 10_000)]
    public int SimulatedLatencyMs { get; set; }
}
```

- **Options pattern:** settings as a **typed class** injected as `IOptions<ProjectStoreOptions>` instead of magic strings.
- `BindConfiguration` (JSON -> object), `ValidateDataAnnotations` (`[Range]`), `ValidateOnStart` (**fail fast** at startup, not on a random request).
- Variants: `IOptions<T>` (read once), `IOptionsSnapshot<T>` (per request, scoped), `IOptionsMonitor<T>` (live changes).

Part 7. ILogger

- `ILogger<T>`: injected; `T` = the **category** (`ProjectHub.Api.Services.ProjectService`) shown on every line; levels can be set per category.
- **Levels:** Trace < Debug < Information < Warning < Error < Critical. `Logging:LogLevel` sets the minimum per category: `Default: Information` (our logs), `Microsoft.AspNetCore: Warning` (quiet framework).
- **Message templates, not interpolation:** `LogInformation("Created project {ProjectId}", id)` makes `ProjectId` a **searchable field** in log systems; `$"..."` bakes it into text. A template isn't even formatted if the level is off.
- `NullLogger<ProjectService>.Instance` writes nothing: the optional constructor parameter lets demos/tests skip logging.
- Logs are written **after** the `lock` (topic 4: keep critical sections short).

ProjectService.CreateAsync with logging:

```
public async Task<Project> CreateAsync(string name, string? description, CancellationToken cancellationToken = default)
{
    // Validate the input first: no point "calling the database" with a bad request.
    var trimmedName = ValidateName(name);

    await SimulateDatabaseCallAsync(cancellationToken);

    Project project;
    lock (_gate)
    {
        // Check AND add inside the same lock: otherwise two concurrent calls with the same name
        // could both pass the check before either one adds (a "check-then-act" race).
        EnsureNameIsFree(trimmedName, exceptProjectId: null);

        project = new Project
        {
            Id = _nextProjectId++,
            Name = trimmedName,
            Description = description,
        };
        _projects.Add(project);
    }

    // Log AFTER the lock: other requests never wait for logging.
    // A message TEMPLATE with named placeholders (not $"..."): ProjectId and ProjectName become searchable fields.
    _logger.LogInformation("Created project {ProjectId} {ProjectName}", project.Id, project.Name);
    return project;
}
```

Part 8. Our code

src/ProjectHub.Api/Program.cs: 1. register services, 2. build the pipeline

```
using System.Text.Json.Serialization;
using Microsoft.Extensions.Options;
using ProjectHub.Api.Configuration;
using ProjectHub.Api.Data;
using ProjectHub.Api.Labs.Lifetimes;
using ProjectHub.Api.Middleware;
using ProjectHub.Api.Services;

var builder = WebApplication.CreateBuilder(args);

// ---- 1. Register services in the DI container ("what can be injected, and how long it lives") ----

builder.Services.AddControllers()
    .AddJsonOptions(options =>
        // Enums as text in JSON: "status": "Todo" instead of "status": 0.
        options.JsonSerializerOptions.Converters.Add(new JsonStringEnumConverter()));
// Learn more about configuring OpenAPI at https://aka.ms/aspnet/openapi
builder.Services.AddOpenApi();

// Settings: the "ProjectStore" section of appsettings*.json -> ProjectStoreOptions.
// ValidateOnStart: a bad value (e.g. -1) stops the app at startup instead of failing on some later request.
builder.Services.AddOptions<ProjectStoreOptions>()
    .BindConfiguration(ProjectStoreOptions.SectionName)
    .ValidateDataAnnotations()
    .ValidateOnStart();

// ONE shared in-memory store for the whole app (Singleton): data created by one request is visible to the next.
// A factory, because ProjectService needs values the container must look up first (options, logger).
builder.Services.AddSingleton<IProjectService>(serviceProvider =>
{
    var options = serviceProvider.GetRequiredService<IOptions<ProjectStoreOptions>>().Value;
    var logger = serviceProvider.GetRequiredService<ILogger<ProjectService>>();
    return new ProjectService(SampleData.CreateProjects(), TimeSpan.FromMilliseconds(options.SimulatedLatencyMs), logger);
});

// Topic 7 lab: the three lifetimes side by side (GET /api/labs/lifetimes).
builder.Services.AddSingleton<SingletonOperation>();
builder.Services.AddScoped<ScopedOperation>();
builder.Services.AddTransient<TransientOperation>();
builder.Services.AddScoped<LifetimeReporter>();

var app = builder.Build();

// ---- 2. The middleware pipeline: every request passes through these, in THIS order ----

// First, so it measures everything that comes after it.
app.UseMiddleware<RequestTimingMiddleware>();

if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
}

app.UseHttpsRedirection();
```

(continued)

```
app.UseAuthorization();

app.MapControllers();

app.Run();
```

- **Why a factory** for `IProjectService`: the constructor needs a `List<Project>` and a `TimeSpan` (not services); the factory asks the container for the options and the logger.
- **Trade-off:** taking `IOptions<ProjectStoreOptions>` in the service would tie it to ASP.NET's options system; a plain `TimeSpan` keeps it easy to create anywhere (demos, tests).

8.1 The lifetimes lab

OperationBase.cs (explicit constructor added after the 6-stops puzzle, Part 10)

```
namespace ProjectHub.Api.Labs.Lifetimes;

// TOPIC 7 LAB: every object gets a random id when it is CREATED. Comparing ids shows whether
// two places received the same object or two different objects from the DI container.
public abstract class OperationBase
{
    // The constructor runs ONLY when an object is created: put a breakpoint here to count creations.
    // (A breakpoint on the Id line would also stop on every READ of Id, because that line holds the getter too.)
    protected OperationBase()
    {
        Id = Guid.NewGuid().ToString()[..8];
    }

    public string Id { get; }
}
```

LifetimeReporter.cs and LifetimesController.cs

```
namespace ProjectHub.Api.Labs.Lifetimes;

// A second consumer in the SAME request, so we can compare what it received with what the controller received.
public class LifetimeReporter(SingletonOperation singleton, ScopedOperation scoped, TransientOperation transient)
{
    public object Report() => new { singleton = singleton.Id, scoped = scoped.Id, transient = transient.Id };
}

using Microsoft.AspNetCore.Mvc;

namespace ProjectHub.Api.Labs.Lifetimes;

// TOPIC 7 LAB: GET /api/labs/lifetimes. Call it twice and compare the ids.
[ApiController]
[Route("api/labs/lifetimes")]
public class LifetimesController(
    SingletonOperation singleton,
    ScopedOperation scoped,
    TransientOperation transient,
    LifetimeReporter reporter) : ControllerBase
{
    [HttpGet]
    public IActionResult Get() => Ok(new
    {
        controller = new { singleton = singleton.Id, scoped = scoped.Id, transient = transient.Id },
        reporter = reporter.Report(),
    });
}
```

Anonymous objects as responses (your question)

`controller = new { singleton = singleton.Id, ... }` creates an **anonymous type** (topic 2), like a JS object literal, nested inside the outer `new { controller = ..., reporter = ... }`. `Ok(...)` serializes it to `{"controller":{"singleton":"ef200b0b",...},"reporter":{...}}`. The compiler generates a class with read-only properties but no name you can write, so it can't be an `ActionResult<T>`. Fine for a lab; real endpoints use **record DTOs** (named contract, OpenAPI can describe it, typed frontend client, testable).

Part 9. Real runs

Cloud container, Development profile (docs/studybooks/notes/07-runs.txt):

```
### lifetimes, request 1
{"controller":{"singleton":"ef200b0b","scoped":"222eb6c4","transient":"08a150dd"},"reporter"
    :{"singleton":"ef200b0b","scoped":"222eb6c4","transient":"ace29ac3"}}
### lifetimes, request 2
{"controller":{"singleton":"ef200b0b","scoped":"34f8a033","transient":"69a01101"},"reporter"
    :{"singleton":"ef200b0b","scoped":"34f8a033","transient":"9d9ee21f"}}
### GET /api/projects (headers)
HTTP/1.1 200 OK
X-Response-Time-Ms: 51
### POST create
HTTP/1.1 201 Created
X-Response-Time-Ms: 123
### POST duplicate
HTTP/1.1 409 Conflict
X-Response-Time-Ms: 77
### POST task
201
### DELETE
204

=== API log (Development, 50 ms latency) ===
warn: Microsoft.AspNetCore.HttpsPolicy.HttpsRedirectionMiddleware[3]
      Failed to determine the https port for redirect.
info: ProjectHub.Api.Middleware.RequestTimingMiddleware[0]
      HTTP GET /api/projects responded 200 in 166 ms
info: ProjectHub.Api.Middleware.RequestTimingMiddleware[0]
      HTTP GET /api/labs/lifetimes responded 200 in 9 ms
info: ProjectHub.Api.Middleware.RequestTimingMiddleware[0]
      HTTP GET /api/labs/lifetimes responded 200 in 1 ms
info: ProjectHub.Api.Middleware.RequestTimingMiddleware[0]
      HTTP GET /api/projects responded 200 in 51 ms
info: ProjectHub.Api.Services.ProjectService[0]
      Created project 4 Docs portal
info: ProjectHub.Api.Middleware.RequestTimingMiddleware[0]
      HTTP POST /api/projects responded 201 in 123 ms
info: ProjectHub.Api.Middleware.RequestTimingMiddleware[0]
      HTTP POST /api/projects responded 409 in 77 ms
info: ProjectHub.Api.Services.ProjectService[0]
      Added task 8 to project 4
info: ProjectHub.Api.Middleware.RequestTimingMiddleware[0]
      HTTP POST /api/projects/4/tasks responded 201 in 68 ms
info: ProjectHub.Api.Services.ProjectService[0]
      Deleted project 4
info: ProjectHub.Api.Middleware.RequestTimingMiddleware[0]
      HTTP DELETE /api/projects/4 responded 204 in 68 ms

### A) environment variable override: ProjectStore__SimulatedLatencyMs=500
HTTP/1.1 200 OK
X-Response-Time-Ms: 601

### B) invalid value: ProjectStore__SimulatedLatencyMs=-1
exit code: 134
      Microsoft.Extensions.Options.OptionsValidationException: DataAnnotation validation
    failed for 'ProjectStoreOptions' members: 'SimulatedLatencyMs' with the error: 'The
    field SimulatedLatencyMs must be between 0 and 10000.'.
```

(continued)

```
Unhandled exception. Microsoft.Extensions.Options.OptionsValidationException: DataAnnotation
    validation failed for 'ProjectStoreOptions' members: 'SimulatedLatencyMs' with the
    error: 'The field SimulatedLatencyMs must be between 0 and 10000.'.

### C) captive dependency (LifetimeReporter registered as singleton)
exit code: 134
Cannot consume scoped service 'ProjectHub.Api.Labs.Lifetimes.ScopedOperation' from singleton
    'ProjectHub.Api.Labs.Lifetimes.LifetimeReporter'.
```

- `X-Response-Time-Ms: 51` = the 50 ms Development latency. Env var 500 -> **601** (the first request also includes warm-up).
- `-1` -> `OptionsValidationException` at startup (exit 134). Captive dependency -> refused to start (exit 134).

Part 10. Debugging

10.1 The 6-stops puzzle (you were right, I was wrong)

With a breakpoint on `public string Id { get; } = Guid.NewGuid()...` you saw **6 stops in both requests**, in one session, with correct lifetime results. I first blamed two debug sessions (the '2' on the debug icon) — **wrong**. You insisted, rightly.

```
public string Id { get; } = Guid.NewGuid().ToString()[..8];
//                 └ getter ┘   └──── initializer (runs once, at creation) ────┘
```

- That line holds the **getter** too: a breakpoint there stops on every **read** of `Id`. Reads per request: controller 3 + reporter 3 = **6**, regardless of creations.
- Your hypothesis ('generation is skipped when not needed') was close; precisely: the id is generated **only at creation**; the extra stops are **reads**.
- Proof via **Frames**: one frame down is `LifetimesController.Get` / `LifetimeReporter.Report` (a read), not the DI container (a creation).
- **Fix:** `OperationBase` got an explicit constructor; a breakpoint inside it counts **creations**: request 1 -> **4** (Singleton lazily on first need, Scoped once, Transient twice), request 2 -> **3** (no Singleton).
- Interview point: **properties are methods** (`{ get; }` compiles to `get_Id()`).
- Lesson for both of us: when an observation contradicts the prediction, check the prediction's assumption (what the breakpoint line actually covers) before blaming the environment.

10.2 The other steps

Step	Result
The middleware onion (breakpoints before/after `next` + in the controller)	order: before `next` -> controller -> after `next`
Break it 1: `AddSingleton<LifetimeReporter>()`	'Some services are not able to be constructed' / 'Cannot consume scoped service ... from singleton' (you did it)
Break it 2: `SimulatedLatencyMs: -1`	the app didn't start: validation (you did it, right)

Part 11. Your answers, corrections, interview Q&A

Question	Your answer	Verdict / complete answer
1. What is DI?	integrating a service via the constructor; composition, easier to maintain (half)	the container creates dependencies and passes them in; benefits: swap implementations, fakes in tests, lifetimes in one place
2. Lifetimes	one per session/runtime; per request; per 'usage' (right)	'usage' = injection
3. Why singleton now / scoped later	shared in-memory global state; DbContext per request (right)	
4. Captive dependency	singleton relying on scoped **or transient** (half)	singleton -> scoped; transient allowed; refuses to start in Development
5. Middleware	Express-like, next, order matters (right)	+ before/after next, short-circuit; timing first
6. OnStarting	?? -> re-check: headers would be gone (right)	headers are sent before the body
7. Config precedence	appsettings -> Development (half) -> re-check: env var wins (right)	double underscore for nesting
8. Options + ValidateOnStart	? -> re-check: typed config + validation, app won't start with a bad value (right)	
9. Templates vs $""	? -> re-check: track/filter by project (right)	searchable field
10. ILogger<T>, levels	?	T = category; minimum level per category in Logging:LogLevel

Interview questions with model answers

Q: What is dependency injection?

Classes declare dependencies in their constructor; the container creates and passes them in. Benefits: swappable implementations, testability with fakes, central lifetime management.

Q: What is the DI container?

A ServiceProvider built from the registrations in Program.cs (builder.Build()); at runtime it creates objects, resolves constructor chains recursively and manages lifetimes.

Q: Singleton vs Scoped vs Transient?

Singleton: one per application process. Scoped: one per HTTP request. Transient: a new instance on every injection.

Q: Which lifetime for a DbContext? For an in-memory store?

DbContext: Scoped. In-memory shared store: Singleton, and it must be thread-safe.

Q: What is a captive dependency?

A longer-lived service holding a shorter-lived one (singleton -> scoped), keeping it forever; ASP.NET refuses to start in Development.

Q: Which classes get constructor injection?

Only classes the container creates: controllers, middleware, registered services. A class you new yourself gets nothing injected.

Q: By-type vs factory registration?

By type: the container calls the constructor and resolves parameters. Factory: you construct it, e.g. when the constructor needs non-service values.

Q: Composition vs inheritance?

Inheritance is is-a (become a subtype); composition is has-a (hold and delegate). Prefer composition: swappable parts, no fragile hierarchies.

Q: What is middleware? Why does order matter?

Components in a pipeline that can act before/after next or short-circuit; each only sees what comes after it.

Q: Why set a response header in OnStarting?

Headers are sent before the body; after the action writes, it can be too late. OnStarting runs right before headers are sent.

Q: How are controllers found?

AddControllers discovers them by reflection (ControllerBase / Controller suffix / attributes); MapControllers maps their attribute routes to endpoints. Folders don't matter.

Q: Configuration precedence?

appsettings.json < appsettings.{Env}.json < user secrets (dev) < environment variables < command line. Nested env vars use __.

Q: What is the Options pattern? ValidateOnStart?

Typed settings classes injected as IOptions<T>; ValidateOnStart validates at startup so bad config fails fast.

Q: Why message templates instead of interpolation in logs?

Placeholders become structured, searchable fields; interpolation produces plain text and is formatted even when the level is off.

Q: What does ILogger<T>'s T do?

Sets the log category (the class name) so you can see the source and configure levels per category.

Q: Why can a breakpoint on a property line stop many times?

An auto-property line holds its getter; every read stops there. Properties are methods.

Part 12. Summary

Key takeaways

DI: register in Program.cs (recipes) -> `builder.Build()` creates the container -> it constructs every class it creates, resolving constructor chains.
Lifetimes: Singleton per app process, Scoped per request, Transient per injection. Never singleton -> scoped (captive dependency).
Composition (has-a) over inheritance (is-a) for your own code; DI plugs the parts in.
Middleware: ordered pipeline, before/after `next`, short-circuit; headers via `OnStarting`.
Controllers are discovered by `AddControllers` and mapped by `MapControllers`; folders don't matter.
Configuration: JSON < environment JSON < secrets < env vars < command line; Options pattern + ValidateOnStart = typed and fail-fast.
Logging: `ILogger<T>` categories, levels, message templates for structured logs; log outside locks.
Debugging: a breakpoint on an auto-property line also stops on reads; count creations in a constructor.

Cheat sheet

Need	Code
one shared instance	`builder.Services.AddSingleton<IFoo, Foo>();`
one per request	`builder.Services.AddScoped<IFoo, Foo>();`
new every time	`builder.Services.AddTransient<IFoo, Foo>();`
custom construction	`builder.Services.AddSingleton<IFoo>(sp => new Foo(sp.GetRequiredService<IBar>(), 42));`
use it	`public class X(IFoo foo)` (constructor injection)
typed settings	`AddOptions<T>().BindConfiguration("Section").ValidateDataAnnotations().ValidateOnStart();` + inject `IOptions<T>`
override a setting	`Section__Key=value` environment variable
middleware	`app.UseMiddleware<MyMiddleware>();` (class with `InvokeAsync(HttpContext)` + `RequestDelegate next`)
log	`logger.LogInformation("Did {Thing} for {Id}", thing, id);`

Checklist: can you do this without notes?

- Explain what the container is and how it builds `LifetimesController`.
- Explain the three lifetimes with the lab output, and the captive-dependency rule with an example.
- Explain why only container-created classes get injection.
- Explain composition vs inheritance with ProjectHub examples.
- Explain middleware order and why `OnStarting` is needed for headers.
- Explain configuration precedence and how to override a setting with an env var.
- Explain message templates vs interpolation in logs.

What comes next: Topic 8 (SQL basics)

SQL Server in Docker, the ProjectHub tables created **by hand** (primary and foreign keys), and ten queries (SELECT, WHERE, ORDER BY, INSERT, UPDATE, DELETE). Checkpoint: query and change data without EF Core.